YASWANTH MUPPALLA

Thinking at the Level of the System

Engineering principle essay for Strange Loop Labs

The Strange Loop principle that resonates most deeply with me is **Think About the System**. The most consequential engineering problems I have worked on were not caused by one broken function or one weak model. They emerged at the boundaries among data, services, users, policies, operational incentives, and failure recovery. Thinking at the system level means treating those boundaries as part of the product. It means asking not only whether a component works, but whether the whole chain produces a reliable outcome for the people who depend on it.

I applied this principle while building AI and data-platform capabilities for credit-oversight and digital-banking users at American Express. A narrow framing would have been to build a retrieval endpoint or connect a language model to a document store. The real problem was larger: heterogeneous source data had to arrive on time, be transformed consistently, remain traceable, respect access controls, and reach an AI workflow with enough context for a grounded response. A useful answer also needed to be explainable after the fact. The system therefore included ingestion, orchestration, retrieval, prompt assembly, tool execution, evaluation, observability, and audit-friendly evidence tracking.

I helped build JSON-configurable Apache Flink pipelines for ingestion, transformation, joins, validation, and downstream publishing, then supported their migration to GCP. I built Airflow workflows with dependency controls, rerun handling, SLA monitoring, data-quality checks, and failure alerts. On top of that foundation, I designed an IRIS-backed RAG ingestion path with extraction, deduplication, upserts, segmentation, chunking, and metadata enrichment. The agent layer used a reusable MCP runtime harness that combined scoped semantic retrieval, tool execution, prompt assembly, source-ledger tracking, and grounded response generation. We managed session state, selected documents, policy context, user intent, and retrieved evidence as explicit parts of the runtime rather than invisible prompt glue.

The system view changed the engineering decisions. A retrieval result without its provenance was incomplete. A fast response without a reliable fallback was not production-ready. A pipeline that succeeded only on the happy path created expensive operational work later. We added retrieval logs, execution traces, citation mapping, guardrails, cache visibility, and audit-friendly response tracking. Those choices contributed to 99% data accuracy, improved reporting efficiency by 35%, and reduced rerun and debugging cycles by three times. The important lesson was that quality did not come from polishing one component; it came from making the interactions among components observable and recoverable.

I learned the same lesson in conventional backend systems. At Skechers, I delivered more than fifteen Python APIs and event-driven services, helped migrate over 35 million records, maintained 99.9% uptime, reduced infrastructure costs by 22%, and cut incident-resolution time by 40%. Those results depended on treating retry behavior, data contracts, alerts, timeouts, and operational ownership as design inputs. The API code mattered, but the surrounding system determined whether the business could trust it.

I have continued developing this principle in OmInsights, my personal technical portfolio project. I designed a six-component applied-AI platform spanning an Angular interface, FastAPI business service, context engine, MCP server, asynchronous file processing, and AWS deployment paths. I built tenant-scoped context flows, a shared LLM gateway with provider routing and fallbacks, source-backed retrieval, and an evaluation layer covering groundedness, relevance, citations, policy checks, and regression tracking. The project reinforced that an AI agent is not a single model call. It is a living system of context, tools, policies, feedback, and operations.

This principle also shapes how I work with stakeholders. Engineers can optimize the wrong subsystem if they do not understand the customer's actual process. I try to map the complete workflow: where information originates, who makes a decision, what delays or rework occur, which failure is tolerable, and which outcome creates value. That shared model helps turn ambiguous requests into small, testable increments while preserving a path toward a reusable platform. It also makes tradeoffs concrete. We can decide where to add rigor, where to prototype, and which field learning should become a product capability.

Strange Loop's operating model connects directly to that approach. Engineers work with customers, build the automation, and carry field knowledge back into a scalable platform. In that environment, thinking about the system is the bridge between a one-off solution and durable leverage. It naturally supports the other principles I value: ship small increments to test assumptions, build quality in before operational load compounds, learn from failures, and turn repeated customer needs into reusable mechanisms.

I want to build AI systems that survive contact with real financial workflows. That requires curiosity about the customer, discipline in architecture, humility about model behavior, and ownership of the full path from source data to measurable outcome. **Think About the System** captures how I have learned to produce trustworthy software - and how I would approach building specialized AI agents at Strange Loop Labs.